Participation

For Special Participation A, I used Grok on the written portion of HW 6.

- Here is the link to my chat:
    - https://grok.com/share/bGVnYWN5_988c8204-0166-4243-9abb-f87b11935144
    - https://grok.com/share/bGVnYWN5_616bb856-339a-4a3d-9303-40914bcf0c55

Analysis

Q2

Overall Assessment

The AI assistant provided a mathematically accurate and conceptually sound response. It correctly identified the core relationship between matrix powers and graph walks and handled the GNN architecture questions well. However, its structure—specifically the separation between "Key Points" and a long narrative essay—made it less direct and harder to follow than the standard "Question-Answer" format found in the satisfactory response.

Strengths (Pros)

1. Strong "Executive Summary" (Key Points)
The AI started with a "Key Points" section. This is a distinct advantage for users who want a high-level overview without wading through the math immediately. It effectively summarized the main takeaways (e.g., A^k counts walks, Max aggregation = Reachability).

2. Mathematical Correctness (Parts a, b, c)
The AI correctly handled the linear algebra and graph theory components.

- **Induction Proof:** The proof for Part (b) was concise and logically sound, correctly identifying the base case and the inductive step involving neighbor summation.
- **Matrix Interpretation:** It correctly identified the output as related to A^k and the update function as the identity.

3. Advanced Domain Knowledge (Parts g, h)

- **Feature Propagation:** In Part (g), the AI mentioned "minimizing Dirichlet energy" and "feature propagation." This is a more advanced, technically specific answer than simply saying "message passing." It shows a deeper knowledge of how modern GNNs handle missing data.
- **Complexity Analysis:** In Part (h), the AI correctly broke down the computational complexity, noting the difference between sparse ($O(|E|)$) and dense ($O(|V|^2)$) graphs.

4. Citations
The AI included a list of citations/sources at the bottom. While this is a standard feature of search-enabled AIs, it adds credibility to the mathematical definitions used in the answer.

Weaknesses (Cons)

1. Structural Redundancy and Poor Formatting
This is the biggest weakness.

- **Split Information:** The AI answers the questions in short bullet points first, but then repeats the entire explanation in a long, narrative essay at the bottom ("Graph neural networks represent...").
- **Hard to Parse:** If a user wants to check the answer for specifically Part (e), they have to look at the short bullet point, and then scan through the long text block at the bottom to see if there is more detail. The "Satisfactory Answer" is much better because it groups the full explanation directly under the question heading.

2. Ambiguity in Notation (Part a)
In Part (a), the AI writes: *"The output... is the j-th row of A^k (or equivalently, the j-th column if considering column vectors)."*
While mathematically true that A is symmetric for undirected graphs, in the context of a specifically defined linear network with one-hot inputs, there is usually one specific correct orientation based on how the multiplication is defined (Ax vs xA). The hedging ("row or column") adds unnecessary cognitive load compared to the specific $A^k e_j$ (column) derivation in the Satisfactory Answer.

3. Weak Analogy in the Table (Part f)
For "Image flip data augmentation," the AI suggests "Graph permutation or subgraph sampling."

- **Critique:** Flipping an image creates a *new* data point. Permuting a graph (renumbering nodes) results in the *exact same* object for a GNN because GNNs are permutation invariant. Therefore, permutation is not really "data augmentation" in the same way an image flip is. The Satisfactory Answer correctly notes that for general graphs, there is no direct equivalent, or suggests geometric rotations for geometric graphs, which is a more precise distinction.

4. Lack of Operational Detail (Part e)
In Part (e-ii) (making the prediction), the AI says: *"Final edge embeddings fed to an MLP..."*
It omits the specific mechanism of selection mentioned in the Satisfactory Answer: **"argmax."** Since the question asks how to predict *which* bond breaks first, explicitly mentioning the selection step (argmax over edge scores) is crucial for a complete answer.

Conclusion

The AI's answer is **correct and trustworthy**, but **poorly organized**. It mimics a blog post or a research summary rather than a homework solution. The "Satisfactory Answer" is superior in terms of pedagogical clarity, as it addresses each sub-problem comprehensively in order, whereas the AI forces the user to piece together the answer from a summary and a dense essay.

Q3

Overall Assessment

The AI assistant provided an **excellent and completely accurate** response. It correctly solved every part of the problem, including the conceptual questions about GNN validity, the mathematical derivation of the loss function, and the structural analysis of the specific update rule. The reasoning was sound throughout, and it successfully interpreted the graph structure provided in the DOT format.

Strengths

1. Perfect Accuracy on Graph Topology (DOT Parsing)

One of the trickier parts of this prompt is interpreting the graph structure defined in the `graph G { ... }` block without a visual image.

- The AI correctly identified the neighbors of Node 2 as $\{1, 4, 5\}$ and Node 3 as $\{5\}$.
- This demonstrates strong logical reasoning capabilities to parse the edge list (`1 -- 2`, `2 -- 5`, etc.) correctly.

2. Detailed and Correct Dimensional Analysis

In Part (c)(ii), the AI provided a step-by-step breakdown of the matrix dimensions.

- Instead of just guessing the answer ($d \times k$), it traced the vector shape through the inner product (W_2), the nonlinearity ($\tanh$), and the aggregation.
- This explicit chain of thought makes the answer very easy to verify and trust.

3. rigorous Conceptual Explanations

In Part (a), the AI clearly distinguished between *valid* and *invalid* update rules by citing the necessary properties: **Permutation Invariance** and **Parameter Sharing**.

- It correctly identified that distinct weights for specific neighbor indices (e.g., w_2 for the 1st neighbor, w_3 for the 2nd) violate the definition of a standard GNN on an unordered graph.

4. Correct Mathematical Formulation

In Part (b), the AI correctly identified the training set vs. the held-out set and applied the binary cross-entropy formula correctly. It did not fall into the trap of including nodes 1 and 4 in the loss calculation.

Weaknesses / Areas for Improvement

1. Mathematical Simplification (Style Preference)

In Part (b), the AI left the final answer in the form of $\log \frac{1}{b}$.

- **AI Answer:** $\frac{1}{3}[\log \frac{1}{b} + \log \frac{1}{c} + \log \frac{1}{1-e}]$
- **Standard Convention:** In Machine Learning, it is more standard to simplify $\log(1/x)$ to $-\log(x)$. The "Satisfactory Answer" uses the negative log form ($\frac{1}{3}(-\log b - \log c \ldots)$), which is generally cleaner and matches how loss functions are typically implemented in code (e.g., `NLLLoss`).
- *Note: The AI's answer is mathematically correct, just stylistically less standard.*

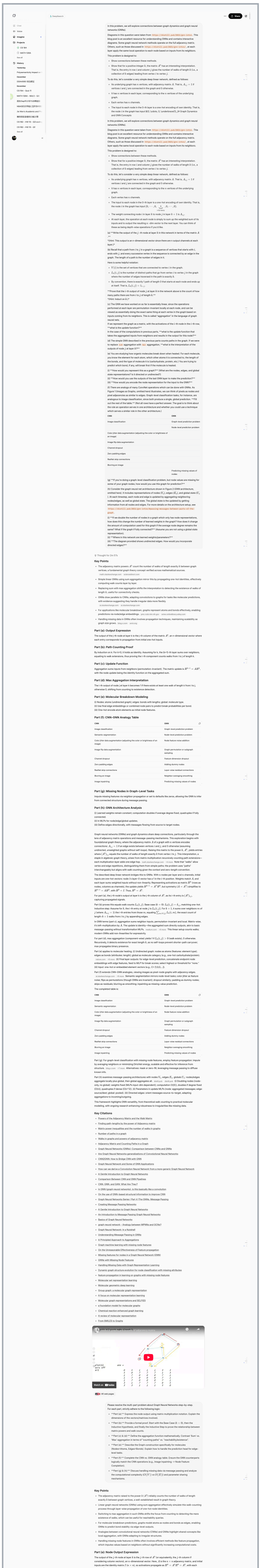
2. Verbosity

The AI's answer is somewhat wordy, particularly in Part (a).

- While the detailed explanations are helpful for a beginner, a more concise expert answer (like the "Satisfactory Answer") gets to the point faster. For example, in (a)(i), the explanation could be condensed to "Valid: Uses symmetric aggregation (mean) and shared weights."

Conclusion

The AI assistant's performance on this problem is **highly reliable**. It made no logical or calculation errors. Its only minor flaw was a lack of algebraic simplification in the final loss term, but this does not affect the correctness of the result. It is a very strong response that would receive full marks in an exam setting.



entry $(A^k)_{i,j}$ in the vector. math.stackexchange.com distill.pub

Part (b): Path Counting Proof

Base Case ($k = 0$): The output is the identity, matching 1 path of length 0 from i to j if $i = j$.

Inductive Hypothesis: Assume for layer k, the i-th entry at node j is $L_k(i, j)$, the walk count.

Inductive Step: At layer $k + 1$, the sum over neighbors yields $\sum_m A_{j,m} L_k(i, m) = L_{k+1}(i, j)$, extending walks by one edge. math.ucsd.edu

Part (c): Update Function

Aggregation: $\sum_{m \in N(j)} h_m^k$, where h_m^k is the input from neighbor m. Matrix form: $H^{k+1} = AH^k$. Update is identity on the aggregate.

Part (d): Max Aggregation Interpretation

Outputs 1 if a walk exists (reachability), 0 otherwise, contrasting sum's quantitative count with max's qualitative existence check. distill.pub

Part (e): Molecular Breakdown Modeling

(i) Nodes: atoms (features: element type); edges: bonds (features: length); global: molecule type; undirected graph.

(ii) Final edge embeddings fed to an MLP for break probability per bond.

(iii) One-hot or learned embeddings for elements. nature.com projects.volkamerlab.org

Part (f): CNN-GNN Analogy Table

CNN	GNN
Image classification	Graph-level prediction problem
Semantic segmentation	Node-level prediction problem
Color jitter data augmentation	Node feature noise addition
Image flip data augmentation	Graph permutation or subgraph sampling
Channel dropout	Feature dimension dropout
Zero padding edges	Adding dummy nodes
ResNet skip connections	Layer-wise residual connections
Blurring an image	Neighbor averaging smoothing
Image inpainting	Predicting missing values of nodes

Part (g): Missing Nodes in Graph-Level Tasks

Use feature propagation to impute missing values by averaging neighbors, allowing message passing to infer from structure. blog.x.com arxiv.org

Part (h): GNN Architecture Analysis

(i) Weights constant; computation $O(|E|)$, doubles with nodes if degree fixed, quadruples if dense.

(ii) In update MLPs for nodes/edges/globals.

(iii) Direct messages from source to target. sassafras13.github.io distill.pub

---

Graph neural networks (GNNs) represent a powerful extension of neural network architectures to handle data structured as graphs, where entities are nodes connected by edges. This framework is particularly useful for problems involving relational data, such as social networks, molecular chemistry, and recommendation systems. At their core, GNNs leverage message passing, where information is exchanged between connected nodes across multiple layers, allowing each node to aggregate and update its representation based on local neighborhoods. This process draws direct parallels to convolutional neural networks (CNNs) on grids but adapts to irregular topologies. neptune.ai +4 more The connection to graph dynamics emerges through the adjacency matrix A, an $n \times n$ matrix where $A_{i,j} = 1$ if nodes i and j are connected (undirected, unweighted graph), and 0 otherwise. Powers of this matrix, A^k, encode the number of walks of length k from i to j in entry $(A^k)_{i,j}$, as matrix multiplication recursively sums paths. math.stackexchange.com +7 more This algebraic property underpins simple GNNs, where layers mimic matrix multiplications.

In the described linear network, each of the n nodes starts with a one-hot n-dimensional input vector, forming the identity matrix $H^0 = I$. Weights align with A, and each layer sums inputs from neighbors, yielding $H^{k+1} = AH^k$, so $H^k = A^k$. For part (a), the output at node j in layer k is the j-th row (or column, depending on convention) of A^k, an $n \times 1$ vector with entries $(A^k)_{*,j}$, where dimensions ensure compatibility: A ($n \times n$) multiplies H^k ($n \times n$) to preserve shape. distill.pub

For part (b), a formal induction proves this equals walk counts $L_k(i, j)$:

- **Base Case** ($k = 0$): $H^0 = I$, so the i-th entry at node j is $\delta_{i,j} = 1$ if $i = j$, else 0, matching the single zero-length path from a node to itself. people.computing.clemson.edu bimsa.net
- **Inductive Hypothesis**: For some $k \geq 0$, the i-th channel of node j's output is $L_k(i, j)$.
- **Inductive Step**: At layer $k + 1$, node j sums inputs from neighbors m where $A_{m,j} = 1$, so the i-th entry becomes $\sum_m A_{m,j} L_k(i, m) = \sum_{m \in N(j)} L_k(i, m)$, which counts all length-$k + 1$ walks by appending one edge from m to j. By induction, it holds for all k. math.stackexchange.com math.ucsd.edu

Part (c) formalizes the GNN as permutation-invariant aggregation at each node: for sum, the aggregate is $\sum_{m \in N(j)} h_m^k$, and the update is the identity function, outputting the sum directly. In matrix terms, this is $H^{k+1} = AH^k$, where H^k stacks node vectors row-wise. distill.pub +5 more For part (d), max aggregation replaces sum with component-wise max: the i-th entry at node j in layer k is 1 if $\max L_k(i, j) > 0$ (i.e., a walk exists), else 0, emphasizing reachability over enumeration—sum quantifies "how many," max answers "if any." distill.pub

In molecular applications (part e), graphs naturally represent structures: (i) Nodes are atoms with features like element type (one-hot encoded, e.g., carbon as [1,0,0,...]); edges are bonds with attributes like length or type; global state encodes molecule category (e.g., carbohydrate via one-hot). The graph is undirected unless chirality requires direction. (ii) For bond-break prediction (edge-level), final layer outputs edge embeddings, concatenated with endpoint nodes and globals, then passed to an MLP for per-edge probabilities; argmax selects the first-breaking bond or "none" if all below threshold. (iii) Initial node features: one-hot elements, possibly augmented with atomic numbers or masses. nature.com +9 more This setup excels in property prediction, outperforming traditional methods by capturing spatial dependencies.

Part (f) completes the CNN-GNN analogy table, illustrating functional parallels despite structural differences—CNNs on Euclidean grids, GNNs on arbitrary graphs. arxiv.org +7 more The table is:

CNN Operation	Role in CNN	GNN Counterpart	Role in GNN
Image classification	Global label from pixel grid	Graph-level prediction	Global property from node/edge features
Semantic segmentation	Per-pixel labels	Node-level prediction	Per-node classifications
Color jitter augmentation	Perturb colors for robustness	Node feature noise	Add Gaussian noise to features
Image flip augmentation	Spatial invariance	Graph permutation/sampling	Relabel nodes or sample subgraphs
Channel dropout	Regularize features	Feature dimension dropout	Mask random dimensions
Zero padding	Handle boundaries	Dummy nodes	Add isolated nodes for padding
ResNet skip connections	Mitigate vanishing gradients	Residual connections	Add skips between GNN layers
Blurring	Smooth features	Neighbor averaging	Mean aggregation smoothing
Image inpainting	Fill missing pixels	Missing node prediction	Impute via propagation

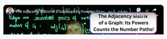
For part (g), in graph-level classification with missing node features, employ feature propagation: minimize Dirichlet energy to impute as weighted neighbor averages, iteratively diffusing known values. This is scalable, $O(|E|)$ per iteration, and integrates seamlessly into message passing without altering the GNN core. blog.x.com +8 more

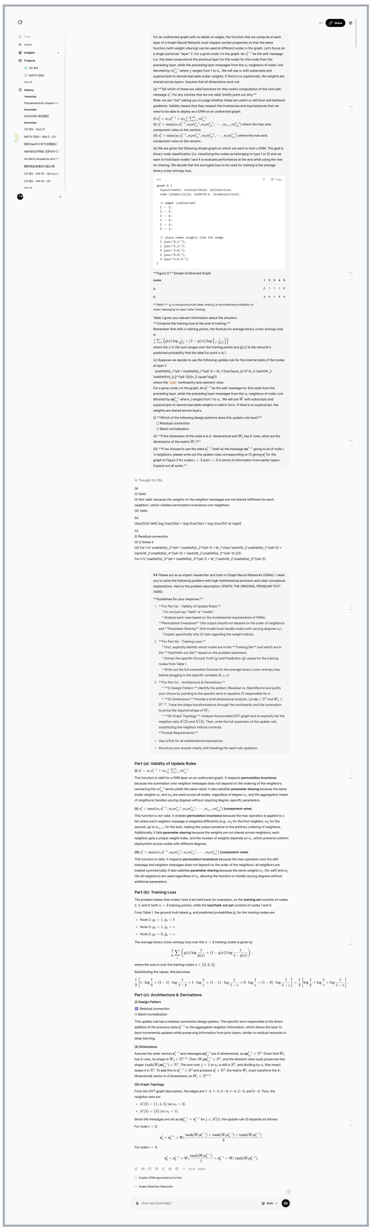
Part (h) analyzes message-passing architectures with nodes V_n, edges E_n, and globals U_n: updates aggregate locals plus globals, then pool to update globals. sassafras13.github.io +10 more (i) Doubling nodes (no globals): parameters unchanged (shared MLPs); computation $O(|E| + |V|)$ for sparse graphs, doubles if average degree constant ($O(|V|)$), but $O(|V|^2)$ quadruples for dense. arxiv.org +8 more (ii) Learned weights are in MLPs for aggregation/update of V_n, E_n, U_n. (iii) For directed edges, messages flow unidirectionally from sources to targets, modifying aggregations to incoming/outgoing.

This comprehensive view underscores GNNs' versatility, from theoretical foundations in linear algebra to practical deployments in chemistry, with ongoing advancements addressing scalability and data imperfections.

Key Citations

- [0] raising adjacency matrix to n-th power gives n-length walks ... - https://math.stackexchange.com/questions/2434064/proof-raising-adjacency-matrix-to-n-th-power-gives-n-length-walks-between
- [1] 9.3: Adjacency Matrices - Engineering LibreTexts - https://eng.libretexts.org/Bookshelves/Computer_Science/Programming_and_Computation_Fundamentals/Mathematics_for_Computer_Science_%28Lehman_Leighton_and_Meyer%29/02%253A_Structures/09%253A_Directed_graphs_and_Partial_Orders/9.03%253A_Adjacency_Matrices
- [2] Graphs as adjacency matrices - Cornell blogs - https://blogs.cornell.edu/info2040/2022/09/03/graphs-as-adjacency-matrices/
- [3] Matrix power inequalities and the number of walks in graphs - https://www.sciencedirect.com/science/article/pii/S0166218X13004174
- [4] [PDF] Matrices and Graphs 12.1 The Adjacency Matrix and Counting ... - https://people.computing.clemson.edu/~goddard/texts/graphTheory/chap12.pdf
- [5] [PDF] Powers of the Adjacency Matrix and the Walk Matrix - https://www.um.edu.mt/library/oar/bitstream/123456789/24439/1/powers%2520of%2520the%2520adjacency%2520matrix.pdf
- [6] The Adjacency Matrix of a Graph and its Powers - YouTube - https://www.youtube.com/watch?v=kc4wBxl5fAg
- [7] [PDF] Chapter 10.3 Counting walks in directed graphs - https://www.math.ucsd.edu/~gptesler/184a/slides/184a_ch10.3slides_17-handout.pdf
- [8] Adjacency Matrices - IB Maths AI Revision Notes - Save My Exams - https://www.savemyexams.com/dp/maths/ib/ai/21/hl/revision-notes/geometry-and-trigonometry/graph-theory/walks-and-adjacency-matrices/
- [9] [PDF] Important Properties and Applications of the Adjacency Matrix ... - https://bimsa.net/doc/notes/48535.pdf?id=0.3935315718408674
- [10] Graph machine learning with missing node features - Blog - https://blog.x.com/engineering/en_us/topics/insights/2022/graph-machine-learning-with-missing-node-features
- [11] Rethinking GNNs and Missing Features: Challenges, Evaluation ... - https://openreview.net/forum?id=gvFOB8VCMr
- [12] On the Unreasonable Effectiveness of Feature propagation in ... - arXiv - https://arxiv.org/abs/2111.12128
- [13] [PDF] Graph Neural Networks with Missing Node Features - Emanuele Rossi - https://emanuelerossi.co.uk/assets/pdf/feature_propagation.pdf
- [14] Missing features for nodes in a Graph Neural Network (GNN) - https://stats.stackexchange.com/questions/637830/missing-features-for-nodes-in-a-graph-neural-network-gnn
- [15] [PDF] Handling Missing Data with Graph Representation Learning - https://papers.neurips.cc/paper_files/paper/2020/file/dc36f18a9a0a776671d4879cae69b551-Paper.pdf
- [16] [PDF] Handling Missing Data with Graph Representation Learning - https://cs.stanford.edu/people/jure/pubs/grape-neurips20.pdf
- [17] Graph convolutional networks for graphs containing missing features - https://www.sciencedirect.com/science/article/pii/S0167739X20330405
- [18] [D] How to handle varying Feature Dimensions in Graph Neural ... - https://www.reddit.com/r/MachineLearning/comments/1h4dbvi/d_how_to_handle_varying_feature_dimensions_in/
- [19] Graph Neural Networks (GNNs): Comparison between CNNs and ... - https://medium.com/%40da-faxi-yuan/graph-neural-networks-gnn-comparison-between-cnn-and-gnn-5c97fdfb3e31
- [20] [PDF] Graph Neural Networks vs Convolutional Neural Networks for ... - arXiv - https://www.arxiv.org/pdf/2511.18150
- [21] [PDF] Comparison Between CNN and GNN Pipelines for Analysing the ... - https://www.scitepress.org/Papers/2025/131735/131735.pdf
- [22] Are Graph Neural Networks generalizations of Convolutional Neural ... - https://ai.stackexchange.com/questions/25367/are-graph-neural-networks-generalizations-of-convolutional-neural-networks
- [23] Graph Neural Network and Some of GNN Applications - https://neptune.ai/blog/graph-neural-network-and-some-of-gnn-applications
- [24] Comparison of CNN and graph neural network (GNN). CNN is ... - https://www.researchgate.net/figure/Comparison-of-CNN-and-graph-neural-network-GNN-CNN-is-applied-to-a-graph-in-Euclidean_fig16_343821039
- [25] A Comprehensive Introduction to Graph Neural Networks (GNNs) - https://www.datacamp.com/tutorial/comprehensive-introduction-graph-neural-networks-gnns-tutorial
- [26] A Gentle Introduction to Graph Neural Networks - Distill.pub - https://distill.pub/2021/gnn-intro
- [27] Graph Convolutional Networks (GCN): From CNN point of view - https://www.youtube.com/watch?v=eLcGehfjvgs
- [28] Graph neural networks: A review of methods and applications - https://www.sciencedirect.com/science/article/pii/S2666651021000012
- [29] Kolmogorov–Arnold graph neural networks for molecular property ... - https://www.nature.com/articles/s42256-025-01087-7
- [30] Chain-aware graph neural networks for molecular property prediction - https://academic.oup.com/bioinformatics/article/40/10/btae574/7818417
- [31] Applying graph neural network models to molecular property ... - https://www.sciencedirect.com/science/article/pii/S2949747724000083
- [32] FragNet: A Graph Neural Network for Molecular Property Prediction ... - https://arxiv.org/abs/2410.12156
- [33] Graph Neural Network-Based Molecular Property Prediction with ... - https://pubs.acs.org/doi/10.1021/acs.jctc.4c00798
- [34] Graph neural networks for materials science and chemistry - PMC - https://pmc.ncbi.nlm.nih.gov/articles/PMC9702700/
- [35] Applying graph neural network models to molecular property ... - https://www.nist.gov/publications/applying-graph-neural-network-models-molecular-property-prediction-using-high-quality
- [36] Graph neural network (GNN) for molecular property ... - GitHub - https://github.com/masashitsubaki/molecularGNN_smiles
- [37] (PDF) Graph Neural Networks for Molecular Property Prediction in ... - https://www.researchgate.net/publication/385627293_Graph_Neural_Networks_for_Molecular_Property_Prediction_in_Drug_Discovery_A_Comprehensive_Review
- [38] T035 - GNN-based molecular property prediction - https://projects.volkamerlab.org/teachopencadd/talktorials/T035_graph_neural_networks.html
- [39] Understanding Message Passing in GNNs - Emma Benjaminson - https://sassafras13.github.io/GNN/
- [40] A Gentle Introduction to Graph Neural Networks - Distill.pub - https://distill.pub/2021/gnn-intro
- [41] Message Passing Neural Networks - Emergent Mind - https://www.emergentmind.com/topics/message-passing-neural-networks-mpnns
- [42] [PDF] GAG: Global Attributed Graph Neural Network for Streaming Session ... - https://arxiv.org/pdf/2007.02747
- [43] Graph Neural Networks Series | Part 4 |The GNNs, Message ... - https://medium.com/the-modern-scientist/graph-neural-networks-series-part-4-the-gnns-message-passing-over-smoothing-e77ffee523cc
- [44] Knowledge-Embedded Message-Passing Neural Networks - https://pubs.acs.org/doi/10.1021/acsomega.1c03839
- [45] [PDF] Efficient Relative Attribute Learning using Graph Neural Networks - https://www.ecva.net/papers/eccv_2018/papers_ECCV/papers/Zihang_Meng_Efficient_Relative_Attribute_ECCV_2018_paper.pdf
- [46] Holistic Graph Neural Networks based on a global-based attention ... - https://www.sciencedirect.com/science/article/abs/pii/S0950705121011667
- [47] A Model-Agnostic Graph Neural Network for Integrating Local and ... - https://pmc.ncbi.nlm.nih.gov/articles/PMC12333528/
- [48] Graph neural networks for materials science and chemistry - Nature - https://www.nature.com/articles/s43246-022-00315-6
- [49] The Computational Complexity of Graph Neural Networks explained - https://medium.com/%40lippoldt331/the-computational-complexity-of-graph-neural-networks-explained-64e751a1ef8b
- [50] On the Computational Capability of Graph Neural Networks - arXiv - https://arxiv.org/abs/2501.08444
- [51] [PDF] Time and Space Complexity of Graph Convolutional Networks - https://qdata.github.io/deep2Read/talks-mb2019/Derrick_201906_GCN_complexityAnalysis-writeup.pdf
- [52] Dynamic personalized graph neural network with linear complexity ... - https://www.sciencedirect.com/science/article/abs/pii/S0952197623014756
- [53] [PDF] The Descriptive Complexity of Graph Neural Networks - https://lics.siglog.org/lics23/Slides/s041.pdf
- [54] [PDF] A Comprehensive Survey on Graph Neural Networks - arXiv - https://arxiv.org/pdf/1901.00596
- [55] Graph Neural Networks - IEEE Xplore - https://ieeexplore.ieee.org/document/9779374/
- [56] Training One-Dimensional Graph Neural Networks is NP-Hard - https://openreview.net/forum?id=7BESdFZ7YA
- [57] Accelerating network layouts using graph neural networks - PMC - https://pmc.ncbi.nlm.nih.gov/articles/PMC10030870/
- [58] The Descriptive Complexity of Graph Neural Networks - NASA ADS - https://ui.adsabs.harvard.edu/abs/2023arXiv230304613G/abstract
- [59] A Gentle Introduction to Graph Neural Networks - https://distill.pub/2021/gnn-intro/


The Adjacency Matrix of a Graph and its Powers
The Adjacency Matrix of a Graph: Its Powers Counts the Number Paths!



For an undirected graph with no labels on edges, the function that we compute at each layer of a Graph Neural Network must respect certain properties so that the same function (with weight-sharing) can be used at different nodes in the graph. Let's focus on a single particular "layer" ℓ. For a given node i in the graph, let $s_i^{\ell-1}$ be the self-message (i.e. the state computed at the previous layer for this node) for this node from the preceding layer, while the preceding layer messages from the n_i neighbors of node i are denoted by $m_{i,j}^{\ell-1}$ where j ranges from 1 to n_i. We will use w with subscripts and superscripts to denote learnable scalar weights. If there's no superscript, the weights are shared across layers. Assume that all dimensions work out.

(a) **Tell which of these are valid functions for this node's computation of the next self-message s_i^ℓ. For any choices that are not valid, briefly point out why.**
Note: we are *not* asking you to judge whether these are useful or will have well behaved gradients. Validity means that they respect the invariances and equivariances that we need to be able to deploy as a GNN on an undirected graph.

(i) $s_i^\ell = w_1 s_i^{\ell-1} + w_2 \frac{1}{n_i} \sum_{j=1}^{n_i} m_{i,j}^{\ell-1}$
(ii) $s_i^\ell = \max(w_1 s_i^{\ell-1}, w_2 m_{i,1}^{\ell-1}, w_3 m_{i,2}^{\ell-1}, \cdots, w_{n_i+1} m_{i,n_i}^{\ell-1})$ where the max acts component-wise on the vectors.
(iii) $s_i^\ell = \max(w_1 s_i^{\ell-1}, w_2 m_{i,1}^{\ell-1}, w_2 m_{i,2}^{\ell-1}, \cdots, w_2 m_{i,n_i}^{\ell-1})$ where the max acts component-wise on the vectors.

(b) We are given the following simple graph on which we want to train a GNN. The goal is binary node classification (i.e. classifying the nodes as belonging to type 1 or 0) and we want to hold back nodes 1 and 4 to evaluate performance at the end while using the rest for training. We decide that the surrogate loss to be used for training is the average binary cross-entropy loss.

```
graph G {
  layout=neato; overlap=false; splines=true;
  node [shape=circle, width=0.6, fixedsize=true];

  // edges (undirected)
  1 -- 2;
  1 -- 5;
  2 -- 4;
  1 -- 4;
  2 -- 5;
  5 -- 3;

  // place nodes roughly like the image
  1 [pos="0,1!"];
  2 [pos="2,1!"];
  4 [pos="2,0!"];
  5 [pos="0,0!"];
  3 [pos="4,0.4!"];
}
```

Figure 3: Simple Undirected Graph

nodes	1	2	3	4	5
y_i	0	1	1	1	0
$\hat{y}_i$	a	b	c	d	e

Table 1: y_i is the ground truth label, while $\hat{y}_i$ is the predicted probability of node i belonging to class 1 after training.

Table 1 gives you relevant information about the situation.
Compute the training loss at the end of training.
Remember that with n training points, the formula for average binary cross-entropy loss is
$\frac{1}{n} \sum_x \left(y(x) \log \frac{1}{\hat{y}(x)} + (1 - y(x)) \log \left(\frac{1}{1-\hat{y}(x)} \right) \right)$
where the x in the sum ranges over the training points and $\hat{y}(x)$ is the network's predicted probability that the label for point x is 1.

(c) Suppose we decide to use the following update rule for the internal state of the nodes at layer ℓ.
\mathbf{s}_i^\ell = \mathbf{s}_i^{\ell-1} + W_1 \frac{\sum_{j=1}^{n_i} \tanh(W_2 \mathbf{m}_{i,j}^{\ell-1})}{n_i} \quad \tag{1}
where the `tanh` nonlinearity acts element-wise.
For a given node i in the graph, let $\mathbf{s}_i^{\ell-1}$ be the self-message for this node from the preceding layer, while the preceding layer messages from the n_i neighbors of node i are denoted by $\mathbf{m}_{i,j}^{\ell-1}$ where j ranges from 1 to n_i. We will use W with subscripts and superscripts to denote learnable weights in matrix form. If there's no superscript, the weights are shared across layers.

(i) **Which of the following design patterns does this update rule have?**
☐ Residual connection
☐ Batch normalization

(ii) **If the dimension of the state $\mathbf{s}$ is d-dimensional and W_2 has k rows, what are the dimensions of the matrix W_1?**

(iii) **If we choose to use the state $\mathbf{s}_i^{\ell-1}$ itself as the message $\mathbf{m}^{\ell-1}$ going to all of node i's neighbors, please write out the update rules corresponding to (1) giving $\mathbf{s}_i^\ell$ for the graph in Figure 3 for nodes $i = 2$ and $i = 3$ in terms of information from earlier layers. Expand out all sums.**

Thought for 28s

(a)
(i) Valid.
(ii) Not valid, because the weights on the neighbor messages are not shared (different for each neighbor), which violates permutation invariance over neighbors.
(iii) Valid.

(b)
\frac{1}{3} \left[\log \frac{1}{b} + \log \frac{1}{c} + \log \frac{1}{1-e} \right]

(c)
(i) Residual connection
(ii) d \times k
(iii) For i=2: \mathbf{s}_2^\ell = \mathbf{s}_2^{\ell-1} + W_1 \frac{ \tanh(W_2 \mathbf{s}_1^{\ell-1}) + \tanh(W_2 \mathbf{s}_4^{\ell-1}) + \tanh(W_2 \mathbf{s}_5^{\ell-1}) }{3}
For i=3: \mathbf{s}_3^\ell = \mathbf{s}_3^{\ell-1} + W_1 \tanh(W_2 \mathbf{s}_5^{\ell-1})

Please act as an expert researcher and tutor in Graph Neural Networks (GNNs). I need you to solve the following problem with high mathematical precision and clear conceptual explanations. Here is the problem description: [PASTE THE ORIGINAL PROBLEM TEXT HERE]

Guidelines for your response:

1. **For Part (a) - Validity of Update Rules:**
   * Do not just say "Valid" or "Invalid."
   * Analyze each case based on the fundamental requirements of GNNs: **Permutation Invariance** (the output should not depend on the order of neighbors) and **Parameter Sharing** (the model must handle nodes with varying degrees n_i).
   * Explain specifically why (ii) fails regarding the weight indices.
2. **For Part (b) - Training Loss:**
   * First, explicitly identify which nodes are in the **Training Set** and which are in the **Test/Held-out Set** based on the problem statement.
   * Extract the specific Ground Truth (y) and Prediction ($\hat{y}$) values for the training nodes from Table 1.
   * Write out the full summation formula for the average binary cross-entropy loss before plugging in the specific variables (b, c, e).
3. **For Part (c) - Architecture & Derivations:**
   * **(i) Design Pattern:** Identify the pattern (Residual vs. BatchNorm) and justify your choice by pointing to the specific term in equation (1) responsible for it.
   * **(ii) Dimensions:** Provide a brief dimensional analysis. Let $\mathbf{m} \in \mathbb{R}^d$ and $W_2 \in \mathbb{R}^{k \times d}$. Trace the shape transformations through the nonlinearity and the summation to prove the required shape of W_1.
   * **(iii) Graph Topology:** Analyze the provided DOT graph text to explicitly list the neighbor sets $\mathcal{N}(2)$ and $\mathcal{N}(3)$. Then, write the full expansion of the update rule, substituting the neighbor indices correctly.
   Format Requirements:

- Use LaTeX for all mathematical expressions.
- Structure your answer clearly with headings for each sub-question.

Part (a): Validity of Update Rules

(i) $s_i^\ell = w_1 s_i^{\ell-1} + w_2 \frac{1}{n_i} \sum_{j=1}^{n_i} m_{i,j}^{\ell-1}$

This function is valid for a GNN layer on an undirected graph. It respects **permutation invariance** because the summation over neighbor messages does not depend on the ordering of the neighbors; reordering the $m_{i,j}^{\ell-1}$ terms yields the same result. It also satisfies **parameter sharing** because the same scalar weights w_1 and w_2 are used across all nodes, regardless of degree n_i, and the aggregation (mean of neighbors) handles varying degrees without requiring degree-specific parameters.

(ii) $s_i^\ell = \max(w_1 s_i^{\ell-1}, w_2 m_{i,1}^{\ell-1}, w_3 m_{i,2}^{\ell-1}, \cdots, w_{n_i+1} m_{i,n_i}^{\ell-1})$ **(component-wise)**

This function is not valid. It violates **permutation invariance** because the max operation is applied to a list where each neighbor message is weighted differently (e.g., w_2 for the first neighbor, w_3 for the second, up to w_{n_i+1} for the last), making the output sensitive to the arbitrary ordering of neighbors. Additionally, it fails **parameter sharing** because the weights are not shared across neighbors; each neighbor gets a unique weight index, and the number of weights depends on n_i, which prevents uniform deployment across nodes with different degrees.

(iii) $s_i^\ell = \max(w_1 s_i^{\ell-1}, w_2 m_{i,1}^{\ell-1}, w_2 m_{i,2}^{\ell-1}, \cdots, w_2 m_{i,n_i}^{\ell-1})$ **(component-wise)**

This function is valid. It respects **permutation invariance** because the max operation over the self-message and neighbor messages does not depend on the order of the neighbors; all neighbors are treated symmetrically. It also satisfies **parameter sharing** because the same weights w_1 (for self) and w_2 (for all neighbors) are used regardless of n_i, allowing the function to handle varying degrees without additional parameters.

Part (b): Training Loss

The problem states that nodes 1 and 4 are held back for evaluation, so the **training set** consists of nodes 2, 3, and 5 (with $n = 3$ training points), while the **test/held-out set** consists of nodes 1 and 4.

From Table 1, the ground truth labels y_i and predicted probabilities $\hat{y}_i$ for the training nodes are:

- Node 2: $y_2 = 1, \hat{y}_2 = b$
- Node 3: $y_3 = 1, \hat{y}_3 = c$
- Node 5: $y_5 = 0, \hat{y}_5 = e$

The average binary cross-entropy loss over the $n = 3$ training nodes is given by

$$\frac{1}{n} \sum_x \left(y(x) \log \frac{1}{\hat{y}(x)} + (1 - y(x)) \log \frac{1}{1-\hat{y}(x)} \right),$$

where the sum is over the training nodes $x \in \{2, 3, 5\}$.

Substituting the values, this becomes

$$\frac{1}{3} \left[1 \cdot \log \frac{1}{b} + (1-1) \cdot \log \frac{1}{1-b} + 1 \cdot \log \frac{1}{c} + (1-1) \cdot \log \frac{1}{1-c} + 0 \cdot \log \frac{1}{e} + (1-0) \cdot \log \frac{1}{1-e} \right] = \frac{1}{3} \left[\log \frac{1}{b} + \log \frac{1}{c} + \log \frac{1}{1-e} \right].$$

Part (c): Architecture & Derivations

(i) Design Pattern

☑ Residual connection
☐ Batch normalization

This update rule has a residual connection design pattern. The specific term responsible is the direct addition of the previous state $\mathbf{s}_i^{\ell-1}$ to the aggregated neighbor information, which allows the layer to learn incremental updates while preserving information from prior layers, similar to residual networks in deep learning.

(ii) Dimensions

Assume the state vectors $\mathbf{s}_i^{\ell-1}$ and messages $\mathbf{m}_{i,j}^{\ell-1}$ are d-dimensional, so $\mathbf{m}_{i,j}^{\ell-1} \in \mathbb{R}^d$. Given that W_2 has k rows, its shape is $W_2 \in \mathbb{R}^{k \times d}$. Then, $W_2 \mathbf{m}_{i,j}^{\ell-1} \in \mathbb{R}^k$, and the element-wise $\tanh$ preserves this shape: $\tanh(W_2 \mathbf{m}_{i,j}^{\ell-1}) \in \mathbb{R}^k$. The sum over $j = 1$ to n_i is still in $\mathbb{R}^k$, and dividing by n_i (the mean) keeps it in $\mathbb{R}^k$. To add this to $\mathbf{s}_i^{\ell-1} \in \mathbb{R}^d$ and produce $\mathbf{s}_i^\ell \in \mathbb{R}^d$, the matrix W_1 must transform the k-dimensional vector to d dimensions, so $W_1 \in \mathbb{R}^{d \times k}$.

(iii) Graph Topology

From the DOT graph description, the edges are 1--2, 1--5, 2--4, 1--4, 2--5, and 5--3. Thus, the neighbor sets are:

- $\mathcal{N}(2) = \{1, 4, 5\}$ (so $n_2 = 3$).
- $\mathcal{N}(3) = \{5\}$ (so $n_3 = 1$).

Since the messages are set as $\mathbf{m}_{i,j}^{\ell-1} = \mathbf{s}_j^{\ell-1}$ for $j \in \mathcal{N}(i)$, the update rule (1) expands as follows:

For node $i = 2$:

$$\mathbf{s}_2^\ell = \mathbf{s}_2^{\ell-1} + W_1 \frac{\tanh(W_2 \mathbf{s}_1^{\ell-1}) + \tanh(W_2 \mathbf{s}_4^{\ell-1}) + \tanh(W_2 \mathbf{s}_5^{\ell-1})}{3}.$$

For node $i = 3$:

$$\mathbf{s}_3^\ell = \mathbf{s}_3^{\ell-1} + W_1 \frac{\tanh(W_2 \mathbf{s}_5^{\ell-1})}{1} = \mathbf{s}_3^{\ell-1} + W_1 \tanh(W_2 \mathbf{s}_5^{\ell-1}).$$